PyTorch

Fashion-MNIST

LECTURE 10 | GÉRON CH. 10

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Lecture 9 fitted a network with `MLPClassifier` and found four things it will not let you do:

you want to	you cannot
change what it optimises	there is no argument for it
see a gradient	nothing is exposed
stop mid-epoch, or log per batch	the smallest unit is one pass
move the computation to another device	there is no device to move to

> **ALL FOUR ARE ONE WALL**
>
> `fit()` contains a loop over epochs and batches that you did not write, cannot see and cannot enter. The repair is not a better library. It is owning the loop.

Today: what that loop has to compute, why it can be computed at all, and how to write it.

Today

1. **The mathematics** — backpropagation as reverse-mode automatic differentiation (20 min)
2. What the wrapper costs, measured (15 min)
3. **The method** — tensors, autograd, and the training loop written out in full (40 min)
4. Two bugs that never raise an exception (15 min)

THE MATHEMATICS

Backpropagation as reverse-mode autodiff

20 minutes · examinable

The question

You called `clf.fit()`.

It computed 266,610 partial derivatives, on every batch, 20 times over. *How?*

> **WHY YOU NEED THE ANSWER**
>
> Because the cost of that computation is what decides whether training a network is possible at all — and the reason it is possible is a property of the *direction* in which the chain rule is applied.

Four ways to get a derivative

Method	Cost for P parameters	Exact?
By hand	your afternoon, per architecture	yes, if you are careful
Finite differences	$P + 1$ evaluations	**✗ no — truncation and cancellation**
Symbolic	expression blow-up	yes
✓ Automatic differentiation	**✓ see below**	**✓ to machine precision**

Automatic differentiation is neither symbolic nor numerical. It differentiates the *program*, operation by operation, using the chain rule and nothing else.

The chain rule, as a product of Jacobians

A network is a composition. Write the loss as

$$L = f_3(f_2(f_1(\boldsymbol{\theta}))) \qquad \Longrightarrow \qquad \frac{\partial L}{\partial \boldsymbol{\theta}} = J_3\, J_2\, J_1$$

where J_i is the Jacobian of f_i evaluated at the point the forward pass reached. Every layer contributes one factor.

✓ Matrix multiplication is associative.

Two bracketings, two algorithms

$$\underbrace{(J_3 J_2)\, J_1}_{\text{reverse mode}} \;=\; \underbrace{J_3\, (J_2 J_1)}_{\text{forward mode}}$$

- **Reverse** — start at the output and work back toward the parameters
- **Forward** — start at the parameters and work forward toward the output

✗ Same number. Wildly different cost.

Forward mode · propagate a tangent

Pick a direction $\mathbf{v}$ in *input* space. Carry $\dot{u} = \partial u / \partial v$ alongside every intermediate value u:

$$\dot{u}_{\text{out}} = \frac{\partial f}{\partial u_{\text{in}}} \dot{u}_{\text{in}}$$

- One sweep gives the derivative of **every output** with respect to **one input direction**
- Nothing needs to be stored: the tangent travels with the value

To get the full gradient with respect to n inputs you run it n times, once per coordinate direction.

Reverse mode · propagate an adjoint

Pick a direction in *output* space — for a scalar loss there is only one, and it is 1. Carry $\bar{u} = \partial L / \partial u$ backwards:

$$\bar{u}_{\text{in}} \;=\; \bar{u}_{\text{out}} \, \frac{\partial f}{\partial u_{\text{in}}}$$

- One sweep gives the derivative of **one output** with respect to **every input**
- The forward values must be kept, because the Jacobians are evaluated at them — that is the memory cost

The cost, in one table

Mode	Sweeps needed	Cheap when
Forward	n — one per **input**	few inputs, many outputs
✓ Reverse	**✓ m — one per output**	**✓ many inputs, few outputs**

Each sweep costs a small constant times one forward evaluation. The constant is around 2 to 4 and does not depend on n or m.

That constant is the reason the backward pass costs about what the forward pass costs, no matter how many parameters there are.

A worked example, small enough to check

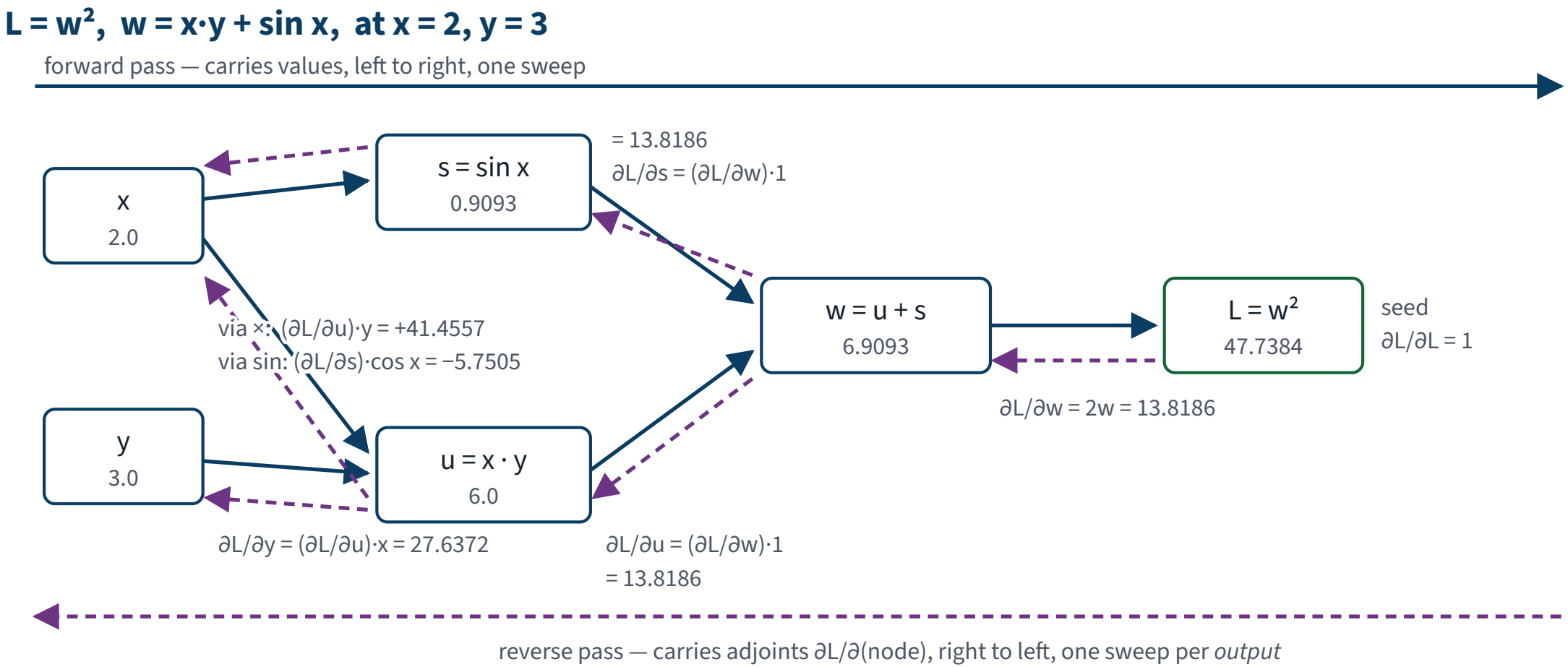
$$L = w^2, \qquad w = xy + \sin x, \qquad \text{at } x = 2,\ y = 3$$

Two inputs, one output. Differentiate it by hand first:

$$\frac{\partial L}{\partial x} = 2w\left(y + \cos x\right), \qquad \frac{\partial L}{\partial y} = 2wx$$

Everything on the next four slides is this function. Small enough to hold in your head; general enough that nothing about the argument changes at 266,610 parameters.

The whole thread, in one picture



One sweep left to right fills in the values. One sweep right to left fills in every derivative.

Two things that graph is showing you

X IS USED TWICE, SO ITS ADJOINT IS A SUM

$\partial L/\partial x = 41.4557 - 5.7505 = 35.7053$. The two paths out of x — through the product and through the sine — each carry a contribution, and the adjoint is their **sum over outgoing edges**. Miss one edge and the gradient is silently wrong, not absent.

ONE REVERSE SWEEP, BOTH DERIVATIVES

Forward mode would need **two** sweeps here: one seeded at x, one at y. Reverse mode costs one sweep per *output*, and there is one output — the loss. That asymmetry is the entire reason training a network with a million parameters and one scalar loss is affordable.

The forward sweep · values

Node	Expression	Value
x	input	2.0
y	input	3.0
u	$x \cdot y$	6.0000
s	$\sin x$	0.9093
w	$u + s$	6.9093
✓ L	✓ w^2	**47.7384** ✓

Six nodes, evaluated once each, in topological order. This is exactly what running the model does — and PyTorch records the graph while it happens.

The reverse sweep · adjoints

Adjoint	Computed as	Value
$\partial L/\partial L$	the seed	1
$\partial L/\partial w$	$2w$	13.8186
$\partial L/\partial u$	$(\partial L/\partial w)\cdot 1$	13.8186
$\partial L/\partial s$	$(\partial L/\partial w)\cdot 1$	13.8186
✓ $\partial L/\partial y$	✓ $(\partial L/\partial u)\,x$	**27.6372** ✓
✓ $\partial L/\partial x$	✓ $(\partial L/\partial u)\,y + (\partial L/\partial s)\cos x$	**35.7052** ✓

Six lines, one pass, both derivatives.

The only bookkeeping in the whole method

x is used **twice** — once by the product and once by the sine.

$$\frac{\partial L}{\partial x} = \underbrace{41.4557}_{\text{via } \times} \underbrace{- 5.7505}_{\text{via } \sin} = 35.7052$$

A node's adjoint is the **sum** of the contributions along its outgoing edges.

That is it. Reverse-mode autodiff is a topological sort, a table of local derivatives, and this sum. There is no other idea in it.

Check it against the library

```
>>> x = torch.tensor(2.0, requires_grad=True)
>>> y = torch.tensor(3.0, requires_grad=True)
>>> L = (x * y + torch.sin(x)) ** 2
>>> L.backward()
>>> x.grad.item(), y.grad.item()
(35.705345, 27.637190)
```

By hand: $2w(y + \cos x) = 35.7052$ and $2wx = 27.6372$.

They agree to machine precision. A cheap, decisive check that the library did what the mathematics says — and the same habit as Lecture 2's orthogonality test.

The argument that settles it

COUNT THE INPUTS AND THE OUTPUTS

Training a network differentiates a function from **266,610 parameters** to **one scalar loss**.

Mode	Sweeps
Forward — one per input	✗ 266,610
✓ Reverse — one per output	1 ✓

✓ Reverse mode is not a good choice. It is the only viable one, by a factor of 266,610.

Why the loss is one number

Not a convention, and not an accident of notation.

- Gradient descent needs a **direction of steepest descent**, and that is only defined for a scalar-valued function
- A vector-valued objective has a Jacobian, not a gradient, and no canonical direction to step in

> So the shape of the training problem — many parameters, one number — is fixed by the optimisation, and that shape is precisely the one reverse mode is cheap for. The two facts are not independent.

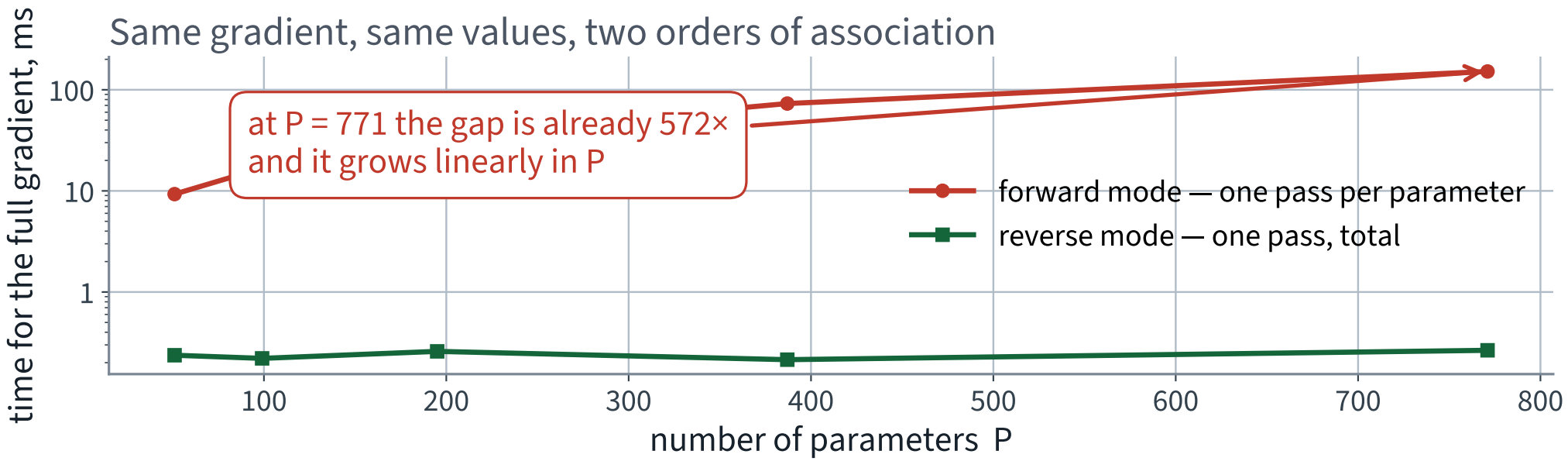
Do not take it on trust — measure it

Small networks, where forward mode can actually be run to completion. Same loss, same point, both modes.

Parameters	Reverse, whole gradient	Forward, whole gradient	Ratio
51	0.24 ms	9.3 ms	39×
99	0.22 ms	16.8 ms	76×
195	0.26 ms	35.5 ms	137×
387	0.21 ms	73.1 ms	341×
771	0.27 ms	151.7 ms	572×

The two columns compute the *same numbers* — they agree to 2.23517e-08 — so this is a measurement of cost, not of quality.

Reverse mode is flat in P; forward mode is linear



✗ Extrapolate the red line to 266,610 parameters. That is why nobody trains networks in forward mode.

At the size we actually train

Quantity	Value	How obtained
Parameters	266,610	counted
Reverse mode, the whole gradient	**1.33 ms ✓**	measured
Forward mode, one direction	0.78 ms	measured
✗ Forward mode, all 266,610	**✗ 3.5 minutes**	**✗ projected, not run**

Two measured per-pass costs and one multiplication you can check. We did not run the last row, and the slide says so — that is the honest form of the claim.

And that is *one* gradient

The previous lecture's run took 20 epochs of 430 batches — 8600 gradients in total, in 125 seconds.

The same 8600 gradients…	Projected wall clock
in forward mode, at the measured per-pass cost	✗ 21 days
✓ in reverse mode	125 seconds, measured ✓

The associativity of matrix multiplication is the only reason this course exists.

So: what is backpropagation?

> **IN ONE SENTENCE**
>
> Reverse-mode automatic differentiation, applied to a neural network, with the seed set to 1 at the loss.

- It is not specific to neural networks
- It is not an approximation
- It is not gradient descent — it *supplies* the gradient that descent then uses

The 1986 paper's contribution was noticing that this was cheap enough to be practical, not inventing the chain rule.

What reverse mode costs

Memory.

Every Jacobian in $J_3 J_2 J_1$ is evaluated at the point the forward pass reached, so every intermediate activation has to be **kept** until the backward pass consumes it.

- Forward mode stores nothing; reverse mode stores the whole forward trace
- The trace grows with the batch size and with the depth

> **WHERE YOU WILL MEET THE BILL**
>
> Lecture 12 computes the RAM of a convolutional network and finds that the activations, not the parameters, are what exhausts the device. This is why.

What the derivation buys you

1. The chain rule is a product of Jacobians, and the product can be bracketed from either end
2. Forward mode costs one sweep per **input**; reverse mode costs one sweep per **output**
3. Training has millions of inputs and **one** output, so reverse mode is the only viable choice — by a factor equal to the parameter count
4. The price is memory: the forward trace must be kept

Point 3 is the one most treatments never state explicitly. It is the whole reason the field works.

MEASURED, NOT ASSERTED

What the wrapper costs

15 minutes

The accuracy is not the bug

88.02% against a 10.0% baseline is a real result.

Unlike Lecture 2, nothing here was measured dishonestly. The split was clean, the validation set was separate, the test set was touched once.

✗ The bug is that you cannot do anything else.

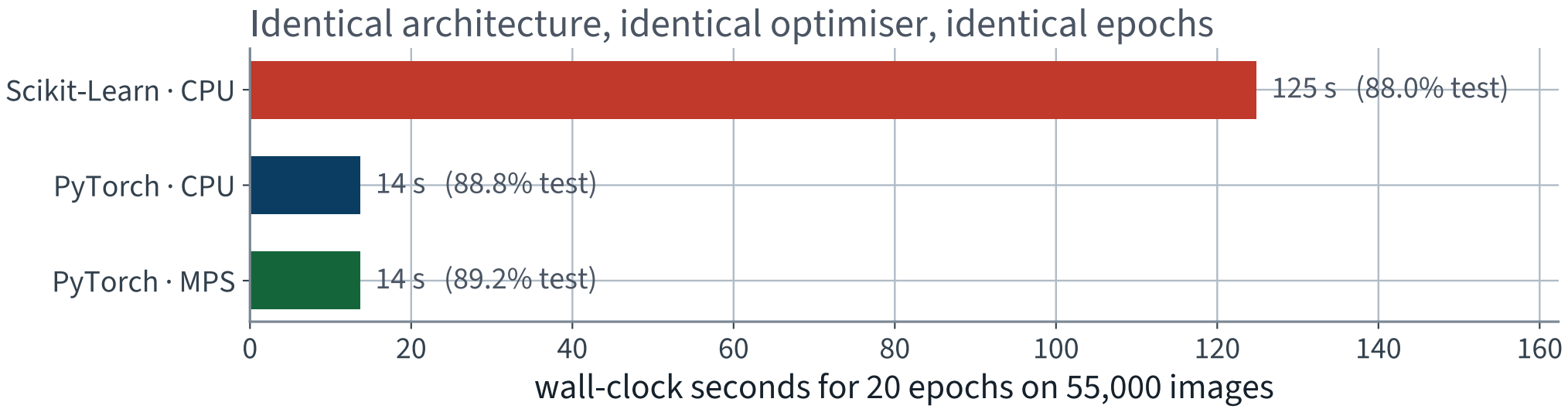
Wall 1 · the clock

Same architecture, same optimiser, same 20 epochs	Wall clock	Test accuracy
Scikit-Learn, CPU	✗ 125 s	88.02%
PyTorch, CPU	14 s	88.84%
✓ PyTorch, MPS	14 s ✓	89.24%

✓ 9.1× faster, for the same model.

✗ Look at rows two and three. The accelerator bought 1.00× — that is, nothing. All of the speed-up is the framework, none of it is the device. Two slides from now we find out why, and it is not a disappointment.

The same problem, three times



The accuracies are within a point of each other. Only the clock moved — and only between the first bar and the other two.

Where the 9.1× came from

Not from the device. From how the arithmetic is dispatched.

- Scikit-Learn's MLP is written for clarity and generality, in NumPy, with a Python-level loop over batches
- PyTorch fuses the same operations into fewer, larger kernels and keeps the tensors in one layout throughout

Same architecture, same optimiser, same epochs, same arithmetic — and 125 seconds against 14, **both on the CPU**.

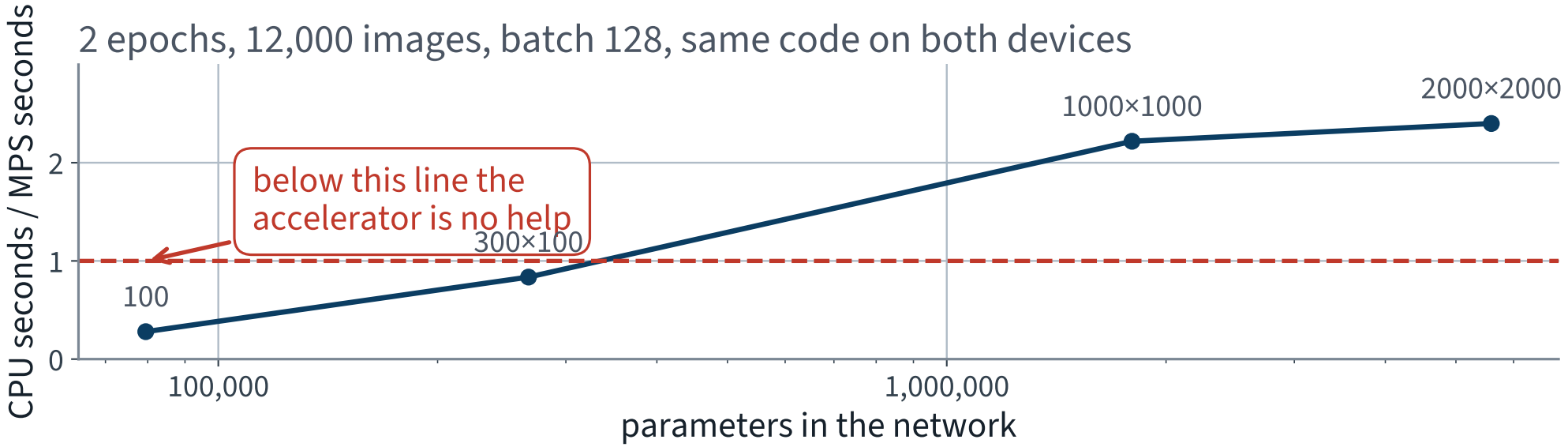
So when does the accelerator pay?

A forward pass is $\phi(\mathbf{X}\mathbf{W} + \mathbf{b})$, repeated, and a GPU is a machine for large matrix products and very little else.

Ours are not large.

266,610 parameters and a batch of 128 is a matrix product that finishes before the cost of dispatching it has been paid. So we measured where the crossover is: same code, same data, widening the layers.

The accelerator earns its place at about a million parameters



✗ At our size the ratio is below 1: the accelerator is *slower*. It only starts winning once each batch carries enough arithmetic to cover the dispatch.

The crossover, measured

Hidden layers	Parameters	CPU	MPS	Ratio
100	79,510	0.13 s	0.45 s	✗ 0.28×
300, 100	266,610	0.21 s	0.25 s	✗ 0.84×
1000, 1000	1,796,010	0.62 s	0.28 s	2.22× ✓
2000, 2000	5,592,010	1.03 s	0.43 s	2.40× ✓

2 epochs on 12,000 images, batch 128, identical code on both devices. Measured on an Apple MPS backend; a Colab T4 has a different crossover, in the same place in the argument.

> **THE HONEST VERSION OF "YOU NEED A GPU"**
>
> You need one when the network is large enough for the device to matter, and not before. Nothing in this course is: every notebook is sized to run on Colab's free CPU tier, deliberately, so that every number on these slides is one you can reproduce. Saying "you need a GPU" without measuring where the crossover falls would be exactly the sort of unmeasured claim this course exists to stop.

Wall 2 · the objective is not yours

```
>>> MLPClassifier(hidden_layer_sizes=(300, 100), loss="mae")
TypeError: MLPClassifier.__init__() got an unexpected
keyword argument 'loss'
```

The operator's actual requirement — some confusions cost more than others — is a class-weighted cross-entropy. One argument in PyTorch. Not reachable at all here.

Wall 3 · nothing is instrumented

Backpropagation computed 266,610 partial derivatives per batch and discarded every one.

THE LECTURE THAT NEEDS THEM

Lecture 11 builds a twenty-layer network whose loss barely moves. The diagnosis is a per-layer gradient statistic, logged during training. With `MLPClassifier` the diagnosis cannot be made at all — you would be left with “it does not train” and no next step.

The diagnosis

> **ONE SENTENCE**
>
> `fit()` contains a loop over epochs and batches that you did not write, cannot see, and cannot enter.

Every wall is the same wall. The repair is therefore not a better library — it is **owning the loop**.

Note what is *not* wrong: the maths, the metric, the split, the architecture. All of that carries over unchanged.

THE METHOD

Tensors, autograd, and the loop

40 minutes · examinable

Today’s notebook

Open Lecture 10 in Google Colab

> **BEFORE YOU RUN ANYTHING**
>
> The notebook prints which device it found in its first cell. Leave it on `cpu`: everything here is sized for the free tier, and the crossover measured two slides ago shows the accelerator running this network *slower*. Change it only to reproduce that measurement.

Tensors

An array that also knows two things a NumPy array does not.

```
1  import torch
2
3  a = torch.tensor([[1., 2.], [3., 4.]])
4  print(a.device)                # cpu
5  b = a.to("cuda")               # or "mps"
6  print(a.requires_grad)         # False — nothing is recorded yet
7  print(a.numpy())               # shares memory with a, on CPU
```

- **Where it lives** — an operation between tensors on different devices is an error, and a loud one
- **Whether to record** — the subject of the next four slides

Finding the device, portably

```
if torch.cuda.is_available():
    device = "cuda"                          # NVIDIA, and Colab
elif torch.backends.mps.is_available():
    device = "mps"                           # Apple Silicon
else:
    device = "cpu"                           # correct, and slow
```

NOT EXAMINABLE

Device selection is engineering, not machine learning. It is on a slide because from here to the end of the course, getting it wrong is the difference between a one-minute cell and a twenty-minute one.

Why `requires_grad` builds a graph

It does not compute a derivative. It says *record what happens to me*.

- Every operation on a recording tensor creates a node holding the operation and its inputs
- The graph is built **during the forward pass**, by running it — there is no separate compilation step
- It is rebuilt from scratch on every batch, which is why a Python `if` inside `forward` just works

That is the forward sweep from thread 6, with the graph written down as it goes so the reverse sweep has something to walk.

You can look at the graph

```
>>> u = torch.tensor(2.0, requires_grad=True)
>>> v = torch.tensor(3.0, requires_grad=True)
>>> q = u * v + torch.sin(u)
>>> r = q ** 2
>>> r.grad_fn
<PowBackward0 object at 0x1291a4b50>
>>> r.grad_fn.next_functions
((<AddBackward0 object at 0x1291a4be0>, 0),)
```

Those are the nodes in the diagram, by name. Walking `next_functions` to the leaves *is* the reverse sweep.

And you can turn it off

```
with torch.no_grad():
    preds = model(X_val)          # no graph is built
```

- Evaluation needs no derivatives, so recording them wastes memory proportional to the size of the batch
- On a large validation set the difference is the difference between fitting on the GPU and not

✗ Forgetting `no_grad` is slow and memory-hungry, but it is *not* a correctness bug. The two bugs that follow are.

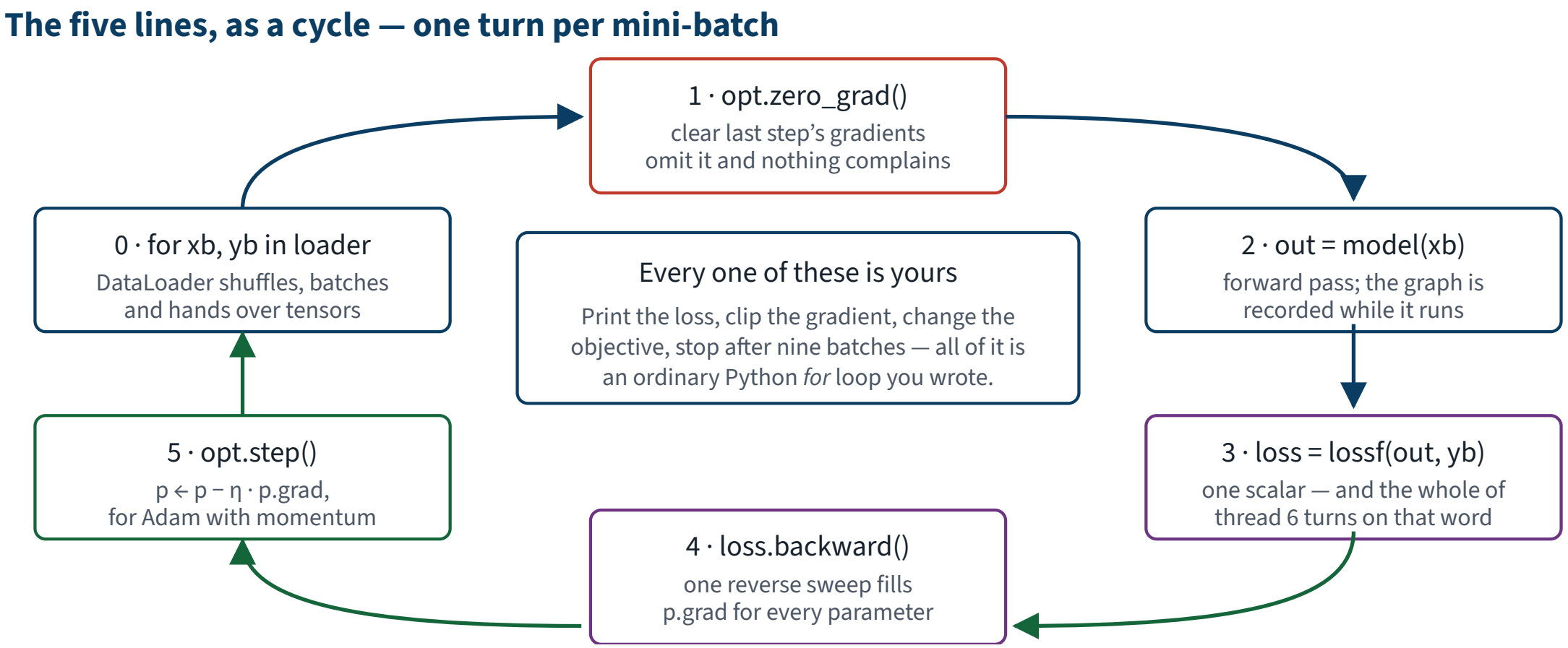
The training loop

```
model = build_model().to(device)
opt   = torch.optim.Adam(model.parameters(), lr=1e-3)
lossf = nn.CrossEntropyLoss()

for epoch in range(EPOCHS):
    model.train()
    for xb, yb in train_loader:
        opt.zero_grad()                    # 1
        out  = model(xb)                   # 2
        loss = lossf(out, yb)              # 3
        loss.backward()                    # 4
        opt.step()                         # 5
```

Five lines. There is no hidden machinery: this is the whole of what `fit()` was doing.

One turn per mini-batch



Every arrow is a line you wrote, so every arrow is a place you can print, clip, log, branch or stop.

Line by line

Line	What it does
`model.train()`	puts dropout and batch-norm layers into training behaviour
✗ **`opt.zero_grad()`**	**✗ clears `p.grad`, which `backward()` *adds* to**
`model(xb)`	the forward sweep; the graph is recorded as it runs
`lossf(out, yb)`	reduces the batch to one scalar — thread 6's whole point
`loss.backward()`	the reverse sweep; fills `p.grad` for every parameter
`opt.step()`	updates the parameters from `p.grad`

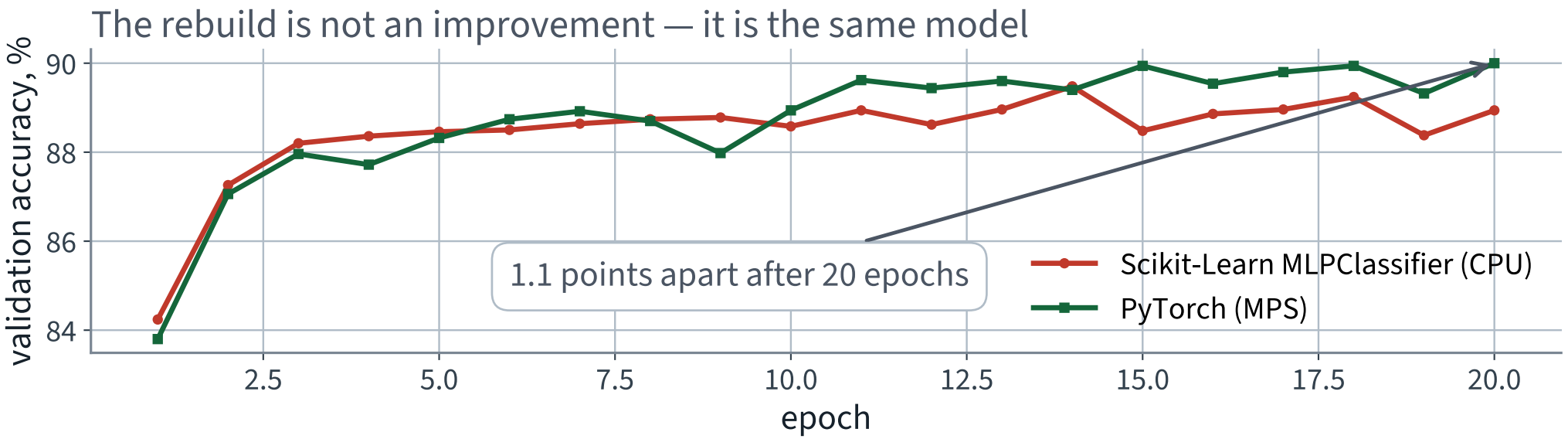
Run it

Model	Test accuracy	Wall clock
Scikit-Learn, CPU	88.02%	125 s
✓ PyTorch, MPS	**89.24% ✓**	**14 s ✓**

The accuracies agree, as they must.

Same architecture, same optimiser, same learning rate, same epochs, same data. **The rebuild is not an improvement — it is the same model.** What we bought is the loop.

Two libraries, one model



If these curves had disagreed, one of the two implementations would be wrong. Overlaying them is the first check after any rewrite.

The first bug that never raises

```
for xb, yb in train_loader:
    out  = model(xb)
    loss = lossf(out, yb)
    loss.backward()
    opt.step()
```

One line has been removed. It runs. The loss goes down.

Take thirty seconds before the next slide.

`backward()` accumulates

`p.grad` is **added to**, not overwritten.

```
after 1 backward call(s): |grad| = 1.6183
after 2 backward call(s): |grad| = 3.2366
after 3 backward call(s): |grad| = 4.8549
after zero_grad():        |grad| = 0.0000
```

That behaviour is deliberate and useful: it is how you accumulate a large effective batch across several small ones, and how you sum gradients from two loss terms. It is only a bug when you did not ask for it.

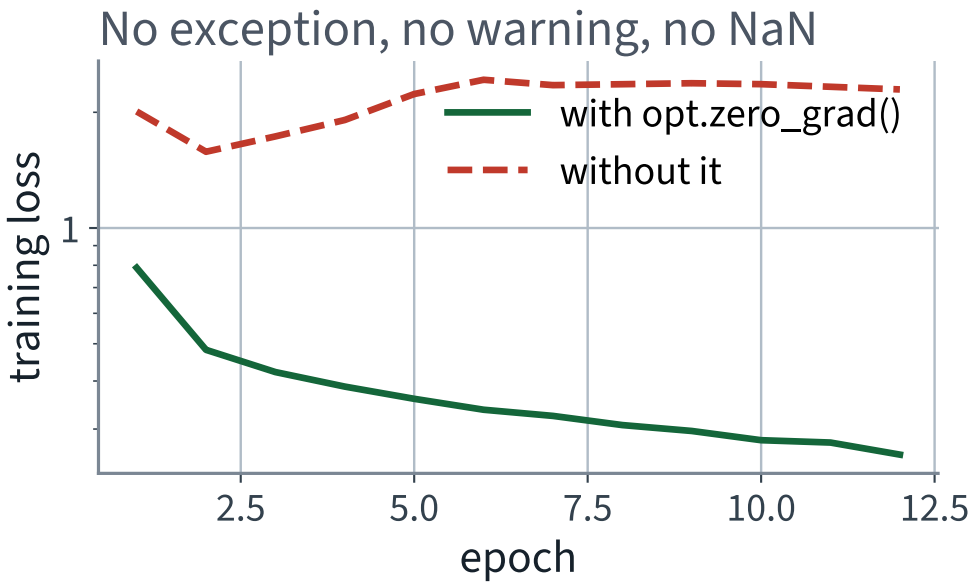
What that does to training

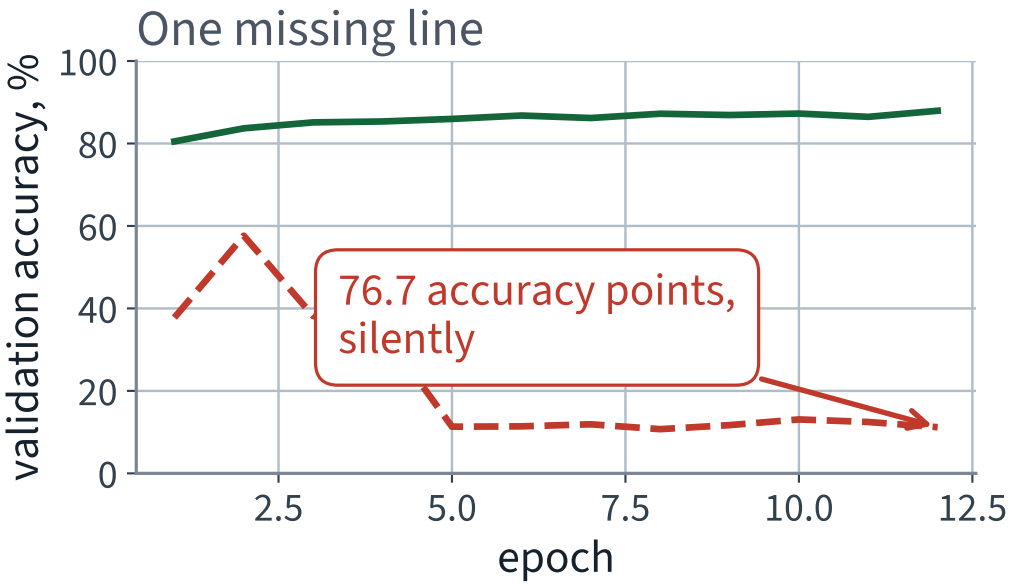
At step t the optimiser sees the sum of every gradient computed so far, not the current one. The effective step size grows without bound, and Adam's normalisation hides the divergence for a while.

✗ No exception. No warning. No `NaN`.

The loss curve does not even look obviously wrong until you put the correct one beside it.

One missing line, measured





✗ **Both runs completed without error. One of them is a working classifier and the other is not.**

The cost of the missing `zero_grad()`

	Final training loss	Validation accuracy
With `opt.zero_grad()`	0.2574	**87.94% ✓**
✗ Without it	**✗ 2.2933**	**✗ 11.20%**

76.7 points

12 epochs on 20,000 images, identical seeds, identical everything else.

How to catch it

1. **Read the loop for the five lines** in order. This is a checklist item, not an insight
2. **Print the gradient norm** for the first few steps. If it climbs monotonically, it is accumulating
3. **Watch where it sits.** `zero_grad()` inside the *epoch* loop instead of the *batch* loop is the same bug, indented two spaces differently, and it survives review

> **THE GENERAL HABIT**
>
> When a framework's default is “accumulate”, the code that does not accumulate is the code with an extra line in it. Ask what happens if that line is deleted — for every line in the loop.

The second bug that never raises

```
model = build_model(dropout=0.2)
train(model, ...)

with torch.no_grad():
    acc = (model(X_test).argmax(1) == y_test).float().mean()
print(f"test accuracy {acc:.4f}")
```

Correct output type, plausible value, no error.

Same question. Thirty seconds.

`train()` and `eval()`

Some layers behave differently in the two phases. Two of them matter in this course:

Layer	In `train()`	In `eval()`
Dropout	zeroes a random fraction of units	passes everything through
Batch normalisation	uses this batch's statistics	uses the running averages

✗ The module defaults to training mode. Build a model, never call `eval()`, and every evaluation you make is of a randomly crippled network.

Batch normalisation arrives in Lecture 11; the same failure applies to it and is harder to see, because it does not fluctuate.

The cost of the missing `model.eval()`

Evaluation mode	Test accuracy
✓ `model.eval()`	**86.77% ✓**
✗ `model.train()`, **mean of 10 calls**	**✗ 86.26%**
lowest of those 10	86.06%
highest of those 10	86.68%

0.51 points

Dropout 0.2, measured on 10,000 test images.

The tell is the spread, not the mean

0.62 accuracy points across 10 identical calls.

> **THE REVIEWER QUESTION THIS GENERATES**
>
> A deterministic function of fixed weights and fixed data returns the same number every time. **If your metric wobbles between calls, ask which layer is still in training mode.**

Run any evaluation twice. It costs one line and it catches this bug, the seeding bugs, and any accidental shuffling of the labels.

What the two bugs have in common

	Missing `zero_grad()`	Missing `eval()`
Raises?	no	no
Warns?	no	no
Produces a plausible number?	✗ yes	✗ yes
Cost, measured today	✗ 76.7 points	✗ 0.51 points
Caught by	reading the loop	running it twice

Both are one line. Both survive a code review that is looking for exceptions rather than for behaviour.

“Running it twice” needs one caveat

Not examinable, and it does not apply to any notebook in this course — they all run on a CPU, where a fixed seed does give the same number twice. It is here because the previous slide told you to run it twice, and the moment you reproduce one of these measurements on an accelerator that stops being true.

A fixed seed does not make a GPU run bit-reproducible. cuDNN picks convolution algorithms by benchmarking them, and reductions that accumulate with atomics add their terms in whatever order the blocks finish. Neither is seeded. Two runs of identical code on identical data differ in about the third decimal.

```
torch.use_deterministic_algorithms(True)
torch.backends.cudnn.deterministic = True   # and benchmark = False
```

Determinism bought back, at a cost in speed — the fastest algorithm is often the non-deterministic one.

✓ So calibrate what counts as evidence: without those flags, a difference in the third decimal means nothing. The two bugs on the previous slide cost 76.7 and 0.51 points — both far outside it.

`Dataset` and `DataLoader`

```
from torch.utils.data import TensorDataset, DataLoader

train_dl = DataLoader(TensorDataset(X_fit, y_fit),
                      batch_size=128, shuffle=True,
                      generator=torch.Generator().manual_seed(42))
```

- `Dataset` answers two questions: how many, and give me number i
- `DataLoader` shuffles, batches, and can load in parallel while the GPU is busy

Indexing a big tensor by hand works while the data fits in memory. It stops working in Lecture 12, and the interface does not change.

The last batch is short

10,000 test images in batches of 384 gives 27 batches, the last containing 16.

> **WHICH MAKES THIS WRONG**
>
> `np.mean([acc(b) for b in batches])` weights the short batch exactly as heavily as a full one.

`drop_last=False` is the default, so nothing is discarded — and that is the right default. The bug is in how the metric is aggregated, not in the loader.

Averaging the metric per batch, measured

How the accuracy was aggregated	Value
✓ Counted over the whole set	89.240% ✓
✗ Mean of the per-batch accuracies	✗ 89.622%

0.382 accuracy points.

Small — *here*. It is small because the last batch is 16 of 384, and because the model is no better or worse on it. Change either and it grows: with batch size 3,000 on 10,000 images the short batch carries a quarter of the weight instead of a twenty-sixth.

The decision rule

> **WEIGHT BY THE BATCH SIZE, OR COUNT OVER THE SET**
>
> Never take a plain mean of per-batch metrics — not because the error is always large, but because **you cannot tell from the number whether it was.**

The identical argument, in the identical shape, as Lecture 1's scale-before-split leak. The damage there was nothing measurable. The rule is procedural for the same reason.

And it is worse for losses than for accuracy: a per-batch mean of a per-batch mean is two averagings deep.

A custom module

```
class Sorter(nn.Module):
    def __init__(self, widths=(300, 100), n_in=784, n_out=10):
        super().__init__()
        sizes = (n_in,) + tuple(widths)
        self.blocks = nn.ModuleList(
            nn.Linear(a, b) for a, b in zip(sizes, sizes[1:]))
        self.head = nn.Linear(sizes[-1], n_out)

    def forward(self, x):
        for blk in self.blocks:
            x = torch.relu(blk(x))
        return self.head(x)
```

`super().__init__()` before anything else, or parameter registration silently does not happen. Everything after that — `.to(device)`, `.parameters()`, `state_dict()` — comes for free.

When `Sequential` runs out

- Two inputs, or two outputs
- A skip connection — the layer's output added to its input
- Anything conditional on the data

All three are ordinary Python inside `forward`, because the graph is rebuilt by *running* the function on every batch.

Skip connections are Lectures 11 and 12. The reason they are easy to write is on this slide.

Evaluation, done properly

```python
@torch.no_grad()
def accuracy(model, X, y, batch=1000):
    model.eval()
    hits = 0
    for i in range(0, len(X), batch):
        out = model(X[i:i + batch])
        hits += (out.argmax(1) == y[i:i + batch]).sum().item()
    return hits / len(X)
```

Three defences in eight lines: no graph, evaluation mode, and a count over the set rather than a mean of means. Write it once, use it everywhere, and all three of today's silent failures become unreachable.

Searching the hyperparameters properly

The previous lecture tuned by hand: ten points, one axis at a time. **Optuna** proposes a configuration, trains it briefly, scores it, and lets a sampler choose the next from what it has learned.

```
def objective(trial):
    lr   = trial.suggest_float("lr", 1e-4, 1e-2, log=True)
    drop = trial.suggest_float("dropout", 0.0, 0.5, step=0.1)
    return validation_accuracy(train(lr=lr, dropout=drop, epochs=6))

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=12)
```

> **NOT EXAMINABLE**
>
> Optuna is not in the book. It is here because it is what you will actually use, and because it makes the point that hand tuning was never a search.

12 trials

Best 86.90% at learning rate 0.00462 and dropout 0.1 — on 6-epoch trials, so this is a search over configurations, not a final model.

Saving the model

```
torch.save(model.state_dict(), "sorter.pt")          # the tensors

fresh = build_model().to(device)
fresh.load_state_dict(torch.load("sorter.pt", weights_only=True))
```

1041 KB — 266,610 float32 numbers and their names.

> **WHAT NOT TO DO**
>
> `torch.save(model, ...)` pickles the object, and unpickling it needs your class definition at the same import path. Next term, when the file has moved, the checkpoint will not load.

And check the reload

```
assert accuracy(model, X_test, y_test) == accuracy(fresh, X_test, y_test)
```

Not “it looks about the same”. A reloaded network is either the same function or it is not, and the difference is one assertion.

The usual cause of failure is loading a `state_dict` into a model built with different widths. PyTorch raises for that — but only if you did not pass `strict=False` to silence it.

THE LAST FIFTEEN MINUTES

Two bugs that never raise an exception

15 minutes

Where we ended

Model	Test accuracy	Wall clock
Always one class	10.0%	—
Scikit-Learn MLP, CPU	88.02%	**✗ 125 s**
✓ PyTorch, same model, MPS	**89.24% ✓**	**14 s ✓**
✗ The same code without `zero_grad()`	**✗ 11.20% (validation)**	unchanged

The last row is the one to remember. It ran, it finished, and it reported a number.

Is that good?

On the operator's own criterion: right 89.2% of the time overall, but only 74.1% on Shirt, with almost all of those errors going to three visually similar queues.

> **WHAT A DEFENSIBLE REPORT SAYS**
>
> Quote the per-class recall, not the headline. Ask whether the three confusable queues are physically adjacent. Offer the operator a choice: an abstain option on the low-confidence cases, or a re-check desk for the three classes that collide.

The model is not the deliverable. The decision the bureau makes with it is.

The silent-failure catalogue so far

Failure	Diagnosed in	Measured cost
Scaler fitted before the split	Lecture 2	nothing measurable
Evaluating on training data	Lecture 2	an RMSE of zero
Accuracy under class imbalance	Lecture 3	a useless model above 90%
Hyperparameters chosen on the test set	Lectures 2, 5	an optimistic report
✗ Missing `optimizer.zero_grad()`	**✗ today**	**✗ 76.7 points**
✗ Missing `model.eval()`	**✗ today**	**✗ 0.51 points**
✗ Metric averaged per batch	**✗ today**	**✗ 0.382 points**

Every one of them ran to completion and printed a plausible number.

PyTorch checklist — keep this

- Is `zero_grad()` inside the batch loop?
- Is `model.train()` set before training, and `model.eval()` before every evaluation?
- Is evaluation wrapped in `torch.no_grad()`?
- Are metrics counted over the set, not averaged over batches?
- Are all tensors on the same device?
- Is the seed fixed — for the model, the loader and the shuffle?
- Does the reloaded checkpoint give exactly the same number?

Seven questions. They will catch most of what goes wrong from here to the end of the course.

What carried over unchanged

From	Still true
Lecture 1	compute the trivial baseline before you believe a score
Lecture 2	split first; never tune on the test set
Lecture 3	count the classes before choosing a metric
Lecture 5	two curves, and the gap between them, beat one number

Nothing about a neural network repeals anything from Part I. The framework changed; the discipline did not.

The notebook for this lecture

Open Lecture 10 in Colab

- **Runs on free CPU.** The corpus is subsampled and the network is small; no accelerator is needed for anything here
- The training loop written out line by line, then the same thing with `DataLoader` and a custom `Module`
- Both silent bugs, reproduced and measured — a missing `zero_grad()` and a missing `model.eval()`

> **WHAT TO CHANGE**
>
> Delete the `optimizer.zero_grad()` line and re-run the loop. Nothing raises. The loss curve is the only thing that tells you, and it is the reason you plot it.

What we did

1. Derived reverse-mode differentiation: the chain rule is a product of Jacobians, the product can be bracketed from either end, and the two ends cost one sweep per input and one sweep per output respectively
2. Concluded that training — millions of parameters, one scalar loss — leaves only one viable bracketing
3. Measured what the convenience wrapper costs, in wall clock and in what it forbids
4. Wrote the training loop: tensors, `requires_grad`, the graph, `backward()`, the optimiser step
5. Replaced it piece by piece with `DataLoader`, a custom `Module`, and a proper evaluation path
6. Met two bugs that raise nothing: gradients accumulating without `zero_grad()`, and dropout still active without `model.eval()`

In the book

Topic	Chapter
Backpropagation, forward and reverse passes	9
Tensors, devices, autograd	10
The training loop, `Dataset` and `DataLoader`	10
Custom modules and the functional API	10
Saving and loading models	10
Dropout	11
Automatic differentiation, in detail	appendix

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 10

five, in the style of the written paper

Exercises — Lecture 10

1 Reverse-mode automatic differentiation costs one sweep for all partial derivatives; forward mode costs one per input. State the condition on the function's shape that makes reverse mode the right choice. [4]

2 Explain why the loss must be a scalar for `.backward()` to be called without arguments. [3]

3 The five lines of the training loop are reordered so that `opt.step()` comes before `loss.backward()`. State what the student observes. [4]

Solutions on Lecture 11's deck.

Exercises — Lecture 10 (continued)

4 A network with dropout is evaluated without calling `model.eval()`. Describe the symptom precisely, and the one-line diagnostic that catches it. [5]

5 GPUs did not help at small model sizes in the measured comparison. Give the reason, and the quantity that decides where the crossover sits. [4]

Solutions on Lecture 11’s deck.

THE FIVE SET AT THE END OF LECTURE 9

Solutions Lecture 9

not part of this lecture

Solutions — Lecture 9

1. A multilayer perceptron with the activation removed is written as $W_3 W_2 W_1 x$. State what function class it…

✓ Only linear maps — the product of matrices is a matrix.

- Composition of linear maps is linear, whatever the depth
- The nonlinearity is what makes an extra layer a new function rather than a re-parameterisation

2. Give the parameter count of a dense layer from m inputs to n units, and evaluate it for the first layer…

✓ $mn + n$; here $784 \times 300 + 300 = 235{,}500$.

- One weight per input-output pair, plus one bias per unit
- The first layer of an image network usually dominates the count, because m is the pixel count

Solutions — Lecture 9 (2)

3. A single TLU cannot represent XOR. State the property of XOR that prevents it, and the smallest change to the…

✓ XOR is not linearly separable; one hidden layer fixes it.

- A TLU's decision boundary is a hyperplane, and no line separates the two XOR classes
- A hidden layer can re-represent the inputs so that the classes become separable in the new coordinates

4. Fashion-MNIST is exactly balanced across ten classes. State the trivial baseline's accuracy, and why…

✓ 10%. With equal class sizes and equal costs, accuracy is not dominated by one class.

- The majority-class predictor scores the largest class share, which is one tenth
- Lecture 3's objection to accuracy was imbalance, and it does not apply

Solutions — Lecture 9 (3)

5. An architecture sweep finds the best hidden-layer size on this dataset. State what that result does and does…

✓ Very little. It holds for this task at the training-set size you swept at, and not even for the same dataset at full size: the architecture the sweep ranked fourth of five is the one that wins on all 55,000 images.

- A deeper network needs more data to pay for its extra parameters, so a sweep run on a fifth of the data systematically prefers the small ones — and never finds out
- It is a measurement of one sweep at one size, not a rule. Reporting it as a general recommendation is the error

LECTURE 11 · GÉRON CH. 11

Training deep networks

Variance propagation through layers, derived — and why a scale error compounds geometrically with depth
Initialisation, normalisation, better optimisers, schedules
A deep stack that will not train, instrumented until it says why

Run the Lecture 10 notebook first